



2.1. Predicados y Auxiliares

Ejercicio 4. Escriba los siguientes predicados auxiliares sobre secuencias de enteros, aclarando los tipos de los parámetros que recibe:

- a) `pred primosEnPosicionesPares(...)` que determina si todos los números primos están en posiciones pares.
- b) `pred enTresPartes(...)`, que determina si en la secuencia aparecen (de izquierda a derecha) primero 0s, después 1s y por último 2s. Por ejemplo $\langle 0, 0, 1, 1, 1, 1, 2 \rangle$ cumple con *enTresPartes*, pero $\langle 0, 1, 3, 0 \rangle$ o $\langle 0, 0, 0, 1, 1 \rangle$ no.
¿Cómo modificaría la expresión para que se admitan cero apariciones de 0s, 1s y 2s (es decir, para que por ejemplo $\langle 0, 0, 0, 1, 1 \rangle$ o $\langle \rangle$ sí cumplan *enTresPartes*)?

Solución

- a) `pred primosEnPosicionesPares(s : seq(Z)) { (∀i : Z) ((0 ≤ i < |s| ∧L esPrimo(s[i])) →L i mód 2 = 0) }`
- b) `pred enTresPartes(s : seq(Z)) { estáOrdenada(s) ∧ sóloCeroUnoYDos(s) }`
donde
`pred sóloCeroUnoYDos(s : seq(Z)) { (∀i : Z) ((0 ≤ i < |s|) →L (s[i] = 0 ∨ s[i] = 1 ∨ s[i] = 2)) }`

2.2. Análisis de especificación

Ejercicio 9. Sea $f : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ definida como:

$$f(a, b) = \begin{cases} 2 \times b & \text{si } a < 0 \\ b - 1 & \text{en otro caso} \end{cases}$$

Indicar cuáles de las siguientes especificaciones son correctas para el problema de calcular $f(a, b)$. Para aquellas que no lo son, indicar por qué.

- a) `proc f(in a, b : R) : R {
 requiere { Verdadero }
 asegura { (a < 0 ∧ res = 2 × b) ∧ (a ≥ 0 ∧ res = b - 1) }
}`
- b) `proc f(in a, b : R) : R {
 requiere { Verdadero }
 asegura { (a < 0 ∧ res = 2 × b) ∨ (a ≥ 0 ∧ res = b - 1) }
}`
- c) `proc f(in a, b : R) : R {
 requiere { Verdadero }
 asegura { (a < 0 → res = 2 × b) ∨ (a ≥ 0 → res = b - 1) }
}`
- d) `proc f(in a, b : R) : R {
 requiere { Verdadero }
 asegura { res = IfThenElse(a < 0, 2 × b, b - 1) }
}`

}

Solución

- a) Es **incorrecta**. No se cumple nunca. Se puede ver que es equivalente a $(P(a) \wedge Q(b)) \wedge (\neg P(a) \wedge R(b)) \equiv (P(a) \wedge \neg P(a)) \wedge Q(b) \wedge R(b)$ que es una contradicción.
- b) Es **correcta**. Es la forma de desglosar un condicional (*if*).
- c) Es **incorrecta**. Es siempre verdadera. Cuando el antecedente de uno es falso la implicación es verdadera y es trivial notar que a es $a < 0$ o $a \geq 0$.
- d) Es **correcta**. Evalúa una condición que depende de a . La rama verdadera ($a < 0$) retorna $2 * b$, mientras que la rama falsa ($a \geq 0$) retorna $b - 1$. En ambos casos se asigna un término a la variable *res* de salida que coincide con lo mostrado para la función partida f .

Ejercicio 10. Considerar la siguiente especificación, junto con un algoritmo que dado x devuelve x^2 .

```
proc unoMasGrande(in x : ℝ) : ℝ {  
  requiere { Verdadero }  
  asegura { res > x }  
}
```

- a) ¿Qué devuelve el algoritmo si recibe $x = 3$? ¿El resultado hace verdadera la postcondición de **unoMasGrande**?
- b) ¿Qué sucede para las entradas $x = 0,5$, $x = 1$, $x = -0,2$ y $x = -7$?
- c) Teniendo en cuenta lo respondido en los puntos anteriores, escribir una **precondición** para **unoMasGrande**, de manera tal que el algoritmo cumpla con la especificación

Solución

- a) Devuelve 9 y hace verdadera la postcondición.
- b) Devuelven 0,25, 1, 0,04 y 49 respectivamente. Los primeros dos no cumplen con la cláusula asegura.
- c) Con pedir que el valor ingresado sea mayor que 1 alcanza: **requiere** $\{|x| > 1\}$

2.3. Relación de fuerza

Ejercicio 12. Considerar las siguientes dos especificaciones, junto con un algoritmo a que satisface la especificación de **p2**.

```
proc p1(in x : ℝ, in n : ℤ) : ℤ {  
  requiere {  $x \neq 0$  }  
  asegura {  $x^n - 1 < res \leq x^n$  }  
}  
  
proc p2(in x : ℝ, in n : ℤ) : ℤ {  
  requiere {  $n \leq 0 \rightarrow x \neq 0$  }  
  asegura {  $res = \lfloor x^n \rfloor$  }  
}
```

- a) Dados valores de x y n que hacen verdadera la precondición de **p1**, demostrar que hacen también verdadera la precondición de **p2**.
- b) Ahora, dados estos valores de x y n , supongamos que se ejecuta a : llegamos a un valor de *res* que hace verdadera la postcondición de **p2**. ¿Será también verdadera la postcondición de **p1** con este valor de *res*?

c) ¿Podemos concluir que a satisface la especificación de $p1$?

Solución

- a) El consecuente de la precondition de $p2$ es la pre de $p1$, por lo tanto, sin importar el valor de n , si la pre de $p1$ es verdadera entonces también lo es la de $p2$
- También podemos pensarlo como dos casos. $n \leq 0$ y $n > 0$. La precondition de $p2$ requiere que en el primer caso $x \neq 0$. Basados en la precondition de $p1$, vemos que esto se cumple para ese caso como para cuando $n > 0$.
- b) Sí, $x^n - 1 < \lfloor x^n \rfloor \leq x^n$ por la definición de $\lfloor x \rfloor$.
- c) Sí, ya que cumple con las condiciones planteadas en el ítem **8.d**.

2.4. Especificación de problemas

2.4.2. Problemas sobre secuencias

Ejercicio 15. Especificar los siguientes problemas sobre secuencias:

- a) Dada una secuencia de números enteros, devolver aquel que divida a más elementos de la secuencia. El elemento tiene que pertenecer a la secuencia original. Si existe más de un elemento que cumple esta propiedad, devolver alguno de ellos.
- b) Dada una secuencia de secuencias de enteros l , devolver una secuencia de l que contenga el máximo valor. Por ejemplo, si $l = \langle \langle 2, 3, 5 \rangle, \langle 8, 1 \rangle, \langle 2, 8, 4, 3 \rangle \rangle$, devolver $\langle 8, 1 \rangle$ o $\langle 2, 8, 4, 3 \rangle$.

Solución

```
a) proc elQueMasDivide(in s : seq<Z>) : Z {
    requiere { Verdadero }
    asegura { res ∈ s ∧ (∀ e : Z) (e ∈ s → divisiblesPor(e, s) ≤ divisiblesPor(res, s)) }

    aux divisiblesPor(n : Z, s : seq<Z>) : Z =  $\sum_0^{|s|-1} \text{IfThenElse}(\text{divideA}(n, s[i]), 1, 0)$ 

    pred divideA(n : Z, m : Z) { n ≠ 0 ∧L m mód n = 0 }
}

b) proc secuenciaMaxima(in l : seq<seq<Z>>) : seq<Z> {
    requiere { Verdadero }
    asegura { res ∈ l ∧L (∃ m : Z) (m ∈ res ∧ esMaximoGeneral(m, l)) }

    pred esMaximoGeneral(m : Z, l : seq<seq<Z>>) { (∀ s : seq<Z>) (s ∈ l → esMaximo(m, s)) }

    pred esMaximo(m : Z, s : seq<Z>) { (∀ n : Z) (n ∈ s → n ≤ m) }
}
```

Ejercicio 17. Dada una secuencia de enteros, se requiere multiplicar por 2 aquéllos valores que se encuentran en posiciones pares. Indicar por qué son incorrectas las siguientes especificaciones y proponer una alternativa correcta.

```
a) proc duplicarPares(inout l : seq<Z>) {
    requiere { l = L0 }
    asegura {
        |l| = |L0| ∧
        (∀ i : Z) (0 ≤ i < |l| ∧ i mód 2 = 0 →L l[i] = 2 × L0[i])
    }
```

```

    }
}

b) proc duplicarPares(inout l : seq(Z)) {
    requiere { l = L0 }
    asegura {
        (∀i : Z) (0 ≤ i < |l| ∧ i mód 2 ≠ 0 →L l[i] = L0[i]) ∧
        (∀i : Z) (0 ≤ i < |l| ∧ i mód 2 = 0 →L l[i] = 2 × L0[i])
    }
}

c) proc duplicarPares(inout l : seq(Z)) : seq(Z) {
    asegura {
        |l| = |res| ∧
        (∀i : Z) (0 ≤ i < |l| ∧ i mód 2 ≠ 0 →L res[i] = l[i]) ∧
        (∀i : Z) (0 ≤ i < |l| ∧ i mód 2 = 0 →L res[i] = 2 × l[i])
    }
}

```

Solución

- a) Se debería especificar que las posiciones impares no se modifican.
- b) Se debería especificar que la longitud de la secuencia no se modifica, en caso contrario, se podrían agregar elementos que no estaban originalmente.
- c) No especifica si se modifica la secuencia l que es parámetro de entrada salida. Debería especificar que el valor contra el que compara res es el valor original de l , usando la metavariante L_0 .
- d) Solución propuesta:
- ```

proc duplicarPares(inout l : seq(Z)) {
 requiere { l = L0 }
 asegura {
 |l| = |L0| ∧L
 (∀i : Z) ((0 ≤ i < |l| ∧ i mód 2 ≠ 0) →L L0[i] = l[i]) ∧
 (∀i : Z) ((0 ≤ i < |l| ∧ i mód 2 = 0) →L L0[i] = 2 * l[i])
 }
}

```

**Ejercicio 18.** Especificar los siguientes problemas de modificación de secuencias:

- a) `proc primosHermanos(inout l : seq(Z))`, que dada una secuencia de enteros mayores a dos, reemplaza dichos valores por el número primo menor más cercano. Por ejemplo, si  $l = \langle 6, 5, 9, 14 \rangle$ , luego de aplicar `primosHermanos(l)`,  $l = \langle 5, 3, 7, 13 \rangle$
- b) `proc reemplazar(inout l : seq(char), in a, b : char)`, que reemplaza todas las apariciones de  $a$  en  $l$  por  $b$

### Solución

- a) `proc primosHermanos(inout l : seq(Z)) {`
- ```

    requiere { l = L0 ∧ (∀i : Z) ((0 ≤ i < |l|) →L (l[i] > 2)) }
    asegura {

```

```

    |l| = |L0| ∧L
    (∀i : ℤ) (0 ≤ i < |l| →L esPrimoMasCercano(l[i], L0[i]))
  }
  pred esPrimoMasCercano(p : ℤ, n : ℤ) { esPrimo(p) ∧ p < n ∧ (∀q : ℤ) ((esPrimo(q) ∧ q < n) → q ≤ p) }
}

b) proc reemplazar(inout l : seq⟨char⟩, in a : char, in b : char) {
  requiere { l = L0 }
  asegura {
    |l| = |L0| ∧L
    (∀i : ℤ) (0 ≤ i < |l| →L (L0[i] = a ∧ l[i] = b) ∨ (L0[i] ≠ a ∧ l[i] = L0[i]))
  }
}

```

2.5. Ejercicios de parciales anteriores

Ejercicio 28. Se desea especificar el problema *primosEnCero* que dada una secuencia s de enteros devuelve la secuencia pero con los valores que se encuentran en posiciones correspondientes a un número primo reemplazados por 0. Por ejemplo,

- $\text{primosEnCero}([0, 1, 2, 3, 4, 5, 6]) = [0, 1, 0, 0, 4, 0, 6]$
- $\text{primosEnCero}([5, 7, -2, 13, -9, 1]) = [5, 7, 0, 0, -9, 0]$

Solución

```

proc primosEnCero(in s : seq⟨ℤ⟩) : seq⟨ℤ⟩ {
  requiere { Verdadero }
  asegura {
    |res| = |s| ∧L (
      primosReemplazados(s, res) ∧
      noPrimosIguales(s, res)
    )
  }
  pred primosReemplazados(s : seq⟨ℤ⟩, res : seq⟨ℤ⟩) {
    (∀i : ℤ) ((0 ≤ i < |s| ∧ esPrimo(i)) →L res[i] = 0)
  }
  pred noPrimosIguales(s : seq⟨ℤ⟩, res : seq⟨ℤ⟩) {
    (∀i : ℤ) ((0 ≤ i < |s| ∧ ¬esPrimo(i)) →L res[i] = s[i])
  }
}

```

Ejercicio 29. Se desea especificar el problema *positivosAumentados* que dada una secuencia s de enteros devuelve la secuencia pero con los valores positivos reemplazados por su valor multiplicado por la posición en que se encuentra.

- $\text{positivosAumentados}([0, 1, 2, 3, 4, 5]) = [0, 1, 4, 9, 16, 25]$
- $\text{positivosAumentados}([-2, -1, 5, 3, 0, -4, 7]) = [-2, -1, 10, 9, 0, -4, 42]$

Solución

```

proc positivosAumentados(in s : seq⟨ℤ⟩) : seq⟨ℤ⟩ {

```

```

requiere { Verdadero }
asegura {
   $|res| = |s| \wedge_L ($ 
     $positivosCambiados(s, res) \wedge$ 
     $negativosIguales(s, res)$ 
   $)$ 
}
pred positivosCambiados( $s : seq\langle \mathbb{Z} \rangle, res : seq\langle \mathbb{Z} \rangle$ ) {
   $(\forall i : \mathbb{Z}) ((0 \leq i < |s| \wedge_L s[i] > 0) \rightarrow_L res[i] = s[i] * i)$ 
}
pred negativosIguales( $s : seq\langle \mathbb{Z} \rangle, res : seq\langle \mathbb{Z} \rangle$ ) {
   $(\forall i : \mathbb{Z}) ((0 \leq i < |s| \wedge_L s[i] \leq 0) \rightarrow_L res[i] = s[i])$ 
}
}

```